

A Novel Application of Ant Colony Optimization to k -defective Maximum Edge Bicliques

Kiyaan Pillai

September 2026

The maximum k -defective edge biclique (k -MDB) problem is an emerging NP-hard optimization. We adapt Ant Colony Optimization (ACO) to this problem and introduce a smaller side preference to outperform previously proposed heuristics. In some instances, ACO produces bicliques with 3-4x more edges. Additionally, ACO provides a tighter upper bound on the size of the k -MDB and can reduce brute force search time by over 90%, though compute constraints limit the robustness of our test. This work examines a novel heuristic approach to the k -MDB problem while identifying several improvements for future heuristics.

1 Introduction

A bipartite graph is a common graph structure where one set of vertices is connected solely to another. Bipartite graphs can model user-product behavior, person-place interaction, and more. A biclique is a set of vertices on a bipartite graph such that each vertex on one side is connected to each vertex on the other. The maximum edge biclique problem aims to identify the biclique with the most edges on a bipartite graph. The problem has applications in online recommendation systems, fraud detection, and community detection [1].

Real-world data often introduces noise to graphs, and structures with relatively few missing connections may still be relevant. A biclique that is allowed up to k missing edges, or k -defective biclique (k -DB), can overcome this real-world noise. Identifying the maximum k -defective edge biclique (k -MDB) remains an emerging field.

The k -MDB problem is NP-hard. Unless $P = NP$, there is no algorithm guaranteed to find the maximum solution that can run in polynomial time. When the absolute optimum is not necessary, approximation techniques are useful to significantly reduce computation while providing meaningfully large bicliques.

Dorigo et al. [2] proposed a metaheuristic for exploring graphs known as Ant Colony Optimization (ACO). Initially used on identifying the shortest path in problems such as the Traveling Salesman Problem, ACO has been adapted to many graph problems, such as vehicle routing [5], quadratic assignment [6],

graph coloring [7], and the maximum clique problem [8], the last of which is closely related to the k -MDB problem via the reduction discussed in Section 2.2. Several improvements have been proposed to the algorithm since its inception, most notably the MAX-MIN Ant System (MMAS) [4]. We adapt MMAS as a heuristic approach to identifying the k -MDB. We observe that it performs significantly faster than brute-force algorithms while still returning comparably large k -defective bicliques.

2 Background

2.1 Definitions

Define a bipartite graph as a graph $G = (U, V, E)$, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges between U and V . Define $n = |U| + |V|$ and $m = |E|$. Let $S = (U_S, V_S)$ denote a subset of vertices where $U_S \subseteq U$ and $V_S \subseteq V$. Let $G(S) = (U_S, V_S, E_S)$ be the induced subgraph of S where $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. Define $\bar{E}(S) = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \notin E\}$ be the set of non-edges in a subgraph. Let a k -defective biclique be a subgraph S where $|\bar{E}(S)| \leq k$.

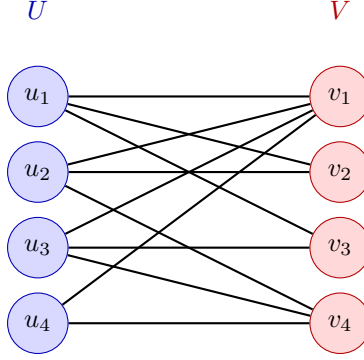


Figure 1: Example bipartite graph $G = (U, V, E)$

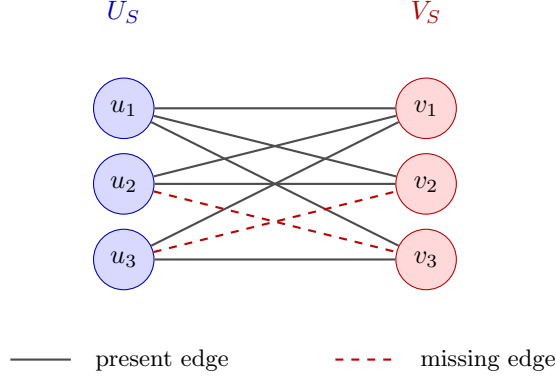


Figure 2: A 2-defective biclique present in Figure 1. Every vertex in U_S is connected to every vertex in V_S except for two missing edges. Since $|\bar{E}(S)| = 2$, this is a k -defective biclique, where $k \geq 2$.

Define the neighbors of a node u in a subgraph S as $N_S(u) = \{v \in S \mid \{u, v\} \in E(S)\}$. Define the nonneighbors of a node as $\bar{N}_S(u) = \{v \in S \mid (u, v) \notin E(S)\}$. Define the degree of a node as $d_S(u) = |N_S(u)|$, and the nondegree as $\bar{d}_S(u) = |\bar{N}_S(u)|$.

Define a maximal k -MDB as a subgraph S that cannot be expanded. That is, for any node $u \notin S$, $|\bar{E}(S \cup \{u\})| > k$. Define the maximum k -MDB as the largest maximal k -MDB in a graph G . Let θ be a user-defined limitation on the solution space. The algorithms proposed in this paper seek to find the maximum k -MDB where both sides have at least θ vertices. This is because, on real-world graphs, k -defective bicliques can be formed with many edges but disproportionate vertex counts, where one side has very few vertices while the other has many [1]. Without the θ term, the utility of the algorithm is limited.

2.2 Computational Complexity

Finding the maximum k -MDB on a bipartite graph is an NP-hard optimization problem. The problem reduces to the well-known NP-hard maximum clique problem, which seeks to find the maximum set of vertices on a graph connected to every other vertex [1].

2.3 Previous Work

Cui et al. [1] proposed a brute-force approach to the k -MDB problem, as well as an initial heuristic designed to find a k -MDB with exactly θ nodes on one side of the graph.

Dorigo et al. [2] proposed a metaheuristic for exploring graphs known as Ant Colony Optimization (ACO). Initially used on identifying the shortest path in problems such as the Traveling Salesman Problem, ACO has been adapted to many graph problems, such as

3 Methods

3.1 ACO Algorithm

ACO traditionally has a set of ants that traverse a graph, leaving pheromone that attracts other ants to follow their path. As the ants explore the solution space, they all converge on one particular path as the quantity pheromones on that path becomes increasingly larger.

We adapt the ACO algorithm to the k -MDB problem. A population of ants iteratively adds vertices from G to a subgraph S , preferring nodes with higher d_S and higher pheromone τ . Here, $\sigma(x) = \frac{1}{1+e^{-x}}$ denotes the standard logistic sigmoid, used to dampen the impact of a node's degree with the entire graph as the size of the subgraph grows.

Algorithm 1 ACO

Ensure: a subset of vertices S such that $G(S)$ is a k -MDB

```

function ACO( $G, N_e, N_S, \rho, T$ )
  for all  $u \in G$  do
     $\tau(u) \leftarrow 1$ 
   $e \leftarrow 0$ 
   $B \leftarrow \{\}$ 
  while  $e < N_e$  do
     $e \leftarrow e + 1$ 
     $S \leftarrow N_S$  empty subgraphs
     $\tau_C \leftarrow \tau$ 
    parallel for each  $S_i \in S$  do
      while  $|\bar{E}(S_i)| < k$  do
         $next \leftarrow$  softmax nodes in  $\{u \in G \mid k \geq |\bar{E}(S \cup \{u\})|\}$  based on
        DESIRABILITY( $u, S_i, \tau_C$ )
        add  $next$  to  $S_i$ 
       $\tau_C(u) \leftarrow \tau_C(u) + T$ 
     $\bar{S}_B \leftarrow \operatorname{argmax}_{S_i \in S} \operatorname{INSTANCEFITNESS}(S_i)$ 
    if  $\operatorname{INSTANCEFITNESS}(\bar{S}_B) > \operatorname{INSTANCEFITNESS}(B)$  then
       $B \leftarrow \bar{S}_B$ 
     $\tau \leftarrow \tau_C$ 
    for all  $u \in G$  do
       $\tau(u) \leftarrow \tau(u) \times \rho$ 
  return  $B$ 

```

We observe that ACO has trouble generating graphs with at least θ nodes because it often adds a disproportionate number of nodes on one side. We first introduce an instance fitness function that rewards a more balanced biclique by squaring the minimum number of nodes. For the desirability function, we introduce a hyperparameter P , a factor that favors nodes on the smaller side when one side of an ant's subgraph has $\geq \theta$ nodes and another side has $< \theta$. We use the following desirability.

Algorithm 2 Prefer-Smaller-Side Desirability

```
function INSTANCEFITNESS( $S$ )
     $a, b \leftarrow \text{minmax}(|U_S|, |V_S|)$ 
    if  $a \geq \theta$  then
        return  $b \times a^2$ 
    else
        return  $a^2$ 
function DESIRABILITY( $u, S, \tau$ )
     $\eta \leftarrow d_S(u) + d_G(u) \times \sigma(|U_S| + |V_S|)$ 
    if  $(|U_S| \geq \theta \oplus |V_S| \geq \theta)$  and  $u$  is on the smaller side then
         $\eta \leftarrow \eta \times P$  ▷ prefer node on the smaller side
    return  $\tau(u) \times \eta^3$ 
```

We attempted an elitist approach to lay additional pheromone on subgraphs with higher fitnesses according to a fitness function, as well as implementing local tabu search on ant subgraphs to prioritize solutions that are more likely to grow. We implemented the MAX-MIN Ant System as well. Finally, we attempted a genetic algorithm approach adapted from Shah’s GCLIQUE framework [3], but found it underperformed both ACO and the θ -heuristic, likely due to the high dimensionality of the encoding required to represent biclique membership across both vertex sets.

3.2 Brute Force Approach

Finally, we implement the approach proposed by Cui et al. These authors observe that many k -MDB have one side with exactly θ vertices and propose a heuristic that removes nodes from one side while adding nodes to another until there are θ nodes on that side. We summarize their algorithm below.

Algorithm 3 θ -Heuristic

Ensure: a k -defective biclique of G

```
function THETAHEURISTIC( $G$ )
     $U \leftarrow \{\}$ 
     $V \leftarrow V_G$ 
    while  $|U| < \theta$  do
        append  $u \in U_G \setminus U$  with largest  $d_V$  to  $U$ 
        while  $|\bar{E}(U \cup V)| > k$  do
            remove  $v \in V$  with largest  $\bar{d}_U$  from  $V$ 
    return  $U \cup V$ 
```

They then propose a branch-and-pivot algorithm that takes advantage of this initial heuristic upper bound. The algorithm checks combinations of $S = (U_S, V_S)$ by maintaining a candidate set $C = (U_C, V_C)$ of all remaining nodes $u \in G$ s.t. $|\bar{E}(S \cup \{u\})| \leq k$ that could still be added to S . It recursively chooses

a node $u \in C$ and branches on $(S' = S \cup \{u\}, C' = \text{candidate set of } S')$ and $(S' = S, C' = C \setminus \{u\})$. Several optimizations are added to make this algorithm faster than a naive brute-force approach. Its time complexity is $O(m\beta_k^n)$, where β is the largest real root of the equation $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. See Appendix B for full pseudocode.

The paper proposes several novel optimization techniques as well. These techniques are beyond the scope of this paper but were implemented during testing. These optimizations include several graph reduction techniques that remove nodes provably not part of a k -MDB.

3.3 Time Complexity

Let $n = |U_G| + |V_G|$ and $m = |E_G|$. For each ant, it must first examine n nodes. At the next step, it will examine $n - 1$ nodes, and then $n - 2$ nodes, etc. Therefore, an ant will examine at most $n + (n - 1) + (n - 2) \dots = \frac{n(n+1)}{2}$. Additionally, at each step, an ant generates a candidate set of nodes that could still be added to its subgraph. This pass is optimized to be $\mathcal{O}(|U_C| + |V_C|)$, where C is the set of candidate nodes before it advanced. If C remains very large, the upper time complexity would therefore be $\mathcal{O}(A \times I \times (n^2 + n))$ for A ants advancing for I iterations, but this would be dominated by n^2 . In a parallelized implementation with p processors, the true upper complexity bound is $\mathcal{O}(\lceil A/p \rceil \times I \times n^2)$.

In practice, however, there are two factors limiting this complexity. First, ants run until they are unable to add more nodes. Let D be the k -MDB of G . Because smaller side preference encourages subgraphs with θ nodes on each side, an ant will empirically visit a maximum of $n_D = |U_D| + |V_D|$ nodes, and therefore have a time complexity of $\mathcal{O}(\lceil A/p \rceil \times I \times n_D \times n)$. n_D is often much lower than the total number of nodes. Additionally, the candidate set is reduced as the ant's subgraph grows. Therefore, the time complexity rarely scales linearly with n .

The θ -heuristic has a time complexity of $\mathcal{O}(\theta n + m)$ [1].

3.4 Experimental Setup

Unless otherwise noted, all ACO experiments use the following hyperparameter values, selected via preliminary tuning on a small subset of graphs:

Table 1: ACO hyperparameters used in all experiments

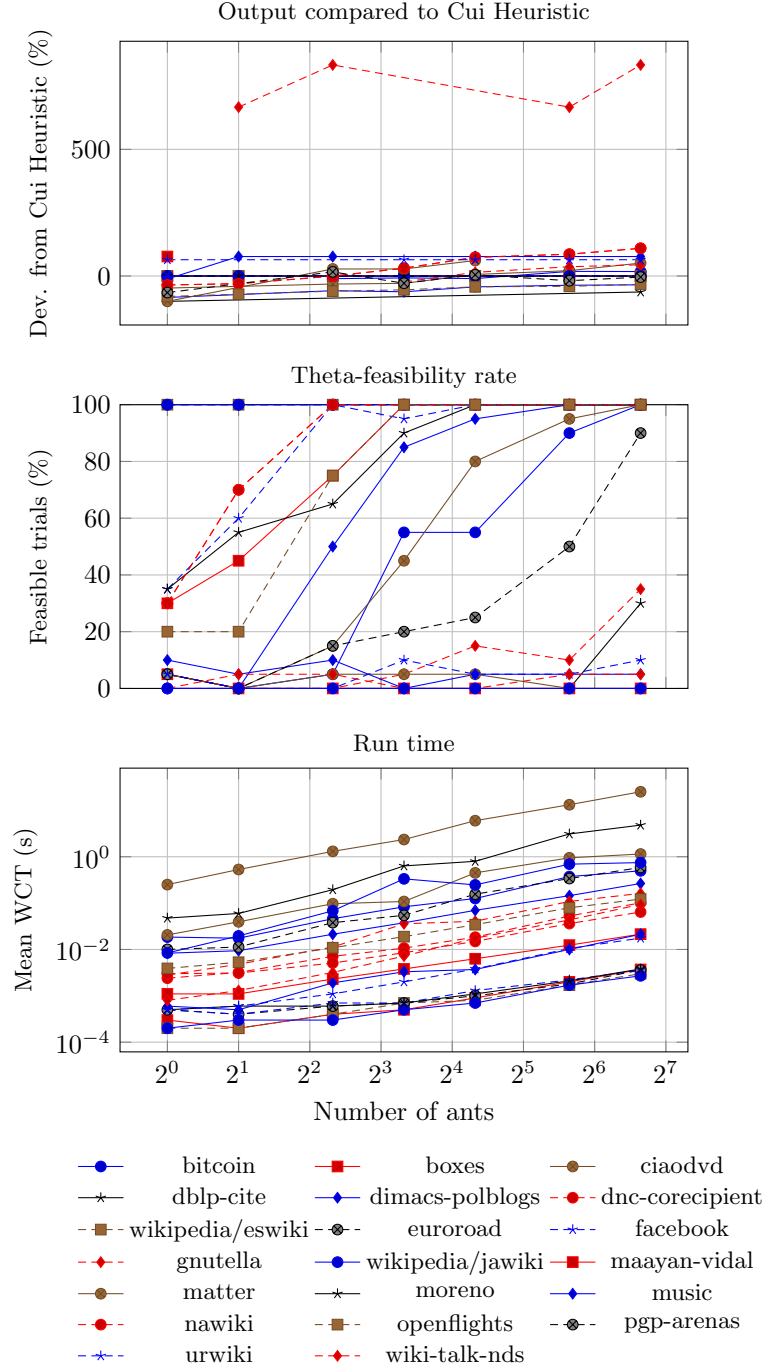
Parameter	Value
Evaporation rate (ρ)	0.95
Pheromone deposit (T)	1
Number of epochs (N_e)	5
Number of ants (N_S)	varied, 1–100 (see below)
Smaller-side preference factor (P)	2
k	2
θ	5

4 Results

We use several graphs from `konect.cc`, a registry of bipartite graphs. We first applied the reductions proposed by Cui et al., and then ran both ACO and the θ -heuristic.

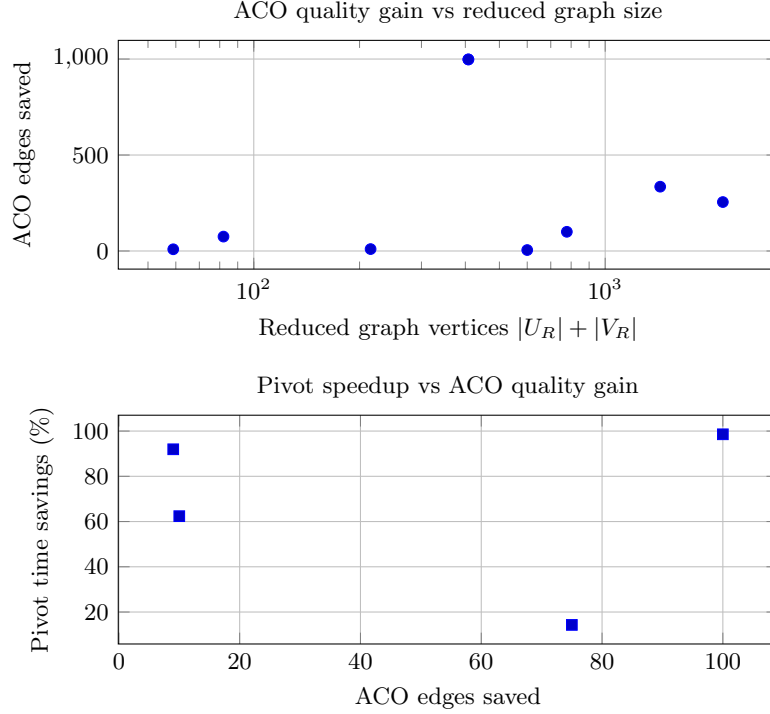
All tests were run with 8 threads on a computer with Ubuntu, an Intel(R) Core(TM) Ultra 7 258V, and 32 gigabytes of RAM.

We also vary the number of ants and analyze the effect on run time as well as solution quality. We compare ACO results to θ -heuristic results. Finally, because Branch and Pivot can prune more branches with a larger initial D^* , we demonstrate the speed increase from using ACO as the heuristic seeding method in cases where ACO outperformed the θ -heuristic. Ant count N_S is varied across $\{1, 2, 5, 10, 20, 50, 100\}$.

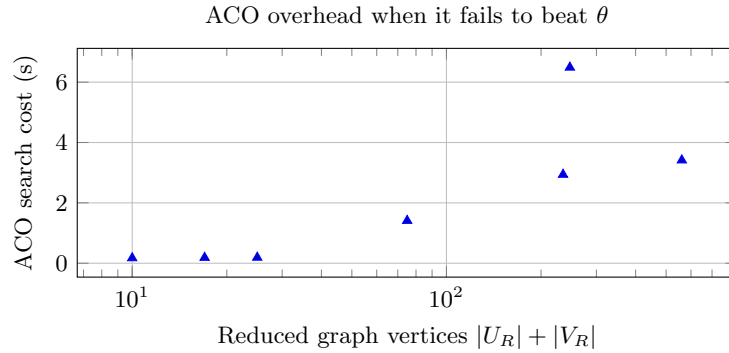


We observe that a marginal increase in the initial upper bound of the subgraph for Branch and Pivot allows the algorithm to prune more branches, and

can sometimes result in significant decreases in runtime. We summarize our results below.



We examined 17 graphs. On 8 of them, ACO did not beat the θ -heuristic and therefore did not yield a seed that could reduce pivot time. Even in those unsuccessful cases, however, the ACO search cost stays modest:



We ran ACO for a total of 5 iterations, and measured both the time to completion as well as the iterations and time to the best returned solution. We measured time and performance for the θ -Heuristic. In no trial did the θ -heuristic find a k -DB with more edges than ACO. On several trials, ACO identifies a much larger k -MDB than the θ -heuristic. Some representative trials are shown below.

5 Discussion

5.1 Interpretation of Results

Across the tested graphs, ACO with smaller-side preference - outperformed theta heuristic, suggesting theta restriction is too restrictive on real-world graphs. Small improvements in

5.2 Limitations

Our evaluation of ACO-seeded branch-and-pivot was constrained to six small graphs (*euroroad*, *facebook*, *kuwiki*, *maayan-vidal*, *moreno*, and *urwiki*). Branch-and-pivot’s runtime grows exponentially with graph size even under a tight bound, and running it to completion on our larger test graphs (e.g., *matter*, *dimacs-polblogs*) was not feasible on the hardware available for this study. As a result, our seeding experiment cannot yet confirm whether the pruning benefit observed on small graphs extends to the graphs where ACO’s advantage over the θ -heuristic is largest (see Section 4).

6 Conclusion

6.1 Future Work

The graphs excluded from our seeded branch-and-pivot experiment are precisely the ones where we would expect the seeding benefit to be largest. On several large graphs — such as *dblp-cite*, *ciaodvd*, and *matter* — ACO found bi-cliques roughly 3–4 $\times$ larger by edge count than the θ -heuristic, at a runtime cost of only a fraction of a second to roughly one second more (Table 1). Because branch-and-pivot’s pruning power depends on the tightness of its initial bound D^* , a substantially larger seed on these graphs should, in principle, prune far more of the search tree than the θ -heuristic’s seed does, potentially yielding much larger runtime reductions than we observed on the small graphs in Section 4. We were unable to test this directly because branch-and-pivot’s runtime on graphs of this size exceeded the resources available for this study. A natural next step is to rerun the seeded comparison on higher-performance hardware to determine whether this relationship holds at scale.

Additionally, ACO could be adapted to have N subspecies that find N solutions in parallel by using a repulsion mechanism similar to that proposed for diversity in other ant colony variants [9], extending previous multi-colony parallelization strategies [10].

7 Works Cited

[1] D. Cui, R. Li, Q. Dai, H. Qin, and G. Wang, “On the Efficient Discovery of Maximum K -Defective Biclique,” arXiv.org. Accessed: Aug. 11, 2026. [Online].

Available: <https://arxiv.org/abs/2506.16121>

- [2] M. Dorigo, V. Maniezzo, and A. Coloni, “Ant System: Optimization by a Colony of Cooperating Agents,” *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996, doi: 10.1109/3477.484436.
- [3] S. Shah, “GCLIQUE: An Open Source Genetic Algorithm for the Maximum Clique Problem,” Aug. 2020, doi: 10.36227/techrxiv.12814217.v1.
- [4] T. Stützle and H. H. Hoos, “– Ant System,” *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, Jun. 2000, doi: 10.1016/S0167-739X(00)00043-1.
- [5] A. E. Rizzoli, R. Montemanni, E. Lucibello, and L. M. Gambardella, “Ant Colony Optimization for Real-World Vehicle Routing Problems,” *Swarm Intelligence*, vol. 1, no. 2, pp. 135–151, Sep. 2007, doi: 10.1007/s11721-007-0005-x.
- [6] L. M. Gambardella, É. D Taillard, and M. Dorigo, “Ant Colonies for the Quadratic Assignment Problem,” *Journal of the Operational Research Society*, vol. 50, no. 2, pp. 167–176, Feb. 1999, doi: 10.1057/palgrave.jors.2600676.
- [7] K. A. Dowsland and J. M. Thompson, “An improved ant colony optimisation heuristic for graph colouring,” *Discrete Applied Mathematics*, vol. 156, no. 3, pp. 313–324, Feb. 2008, doi: 10.1016/j.dam.2007.03.025.
- [8] C. Solnon and S. Fenet, “A Study of ACO Capabilities for Solving the Maximum Clique Problem,” *Journal of Heuristics*, vol. 12, no. 3, pp. 155–180, May 2006, doi: 10.1007/s10732-006-4295-8.
- [9] W. Tfaili and P. Siarry, “A New Charged Ant Colony Algorithm for Continuous Dynamic Optimization,” *Applied Mathematics and Computation*, vol. 197, no. 2, pp. 604–613, Apr. 2008, doi: 10.1016/j.amc.2007.08.087.
- [10] M. Middendorf, F. Reischle, and H. Schmeck, “Multi Colony Ant Algorithms,” *Journal of Heuristics*, vol. 8, no. 3, pp. 305–320, May 2002, doi: 10.1023/a:1015057701750.

8 Appendix A: ACO vs θ -heuristic

Below are the results of running ACO with 100 ants vs the θ -heuristic on 10 different graphs.

9 Appendix B: Branch and Pivot Pseudocode

D^* is the k -MDB of the graph.

Table 2: $k = 2, \theta = 5$

Dataset	$ U_G $	$ V_G $	$ E_G $	$ U_R $	$ V_R $	ACO				θ -Heuristic	
						$ E(D^*) $	Time	ITB	TTB	$ E(D^*) $	Time
Euroroad	—	—	1440	5	5	23	0.0003	1	0.0001	23	0.0000
Moreno	—	—	2300	13	12	23	0.0003	3	0.0001	23	0.0001
Facebook	—	—	3003	9	8	23	0.0003	1	0.0001	23	0.0000
Urwiki	—	—	4018	14	45	76	0.0004	1	0.0001	14	0.0001
Maayan Vidal	—	—	6749	81	134	42	0.0009	1	0.0003	13	0.0003
Dnc Corecipient	—	—	12108	200	208	1426	0.0941	1	0.0299	428	0.0022
Nawiki	—	—	12108	200	208	1426	0.0729	1	0.0228	428	0.0023
Boxes	—	—	16239	14	14	23	0.0002	1	0.0001	23	0.0000
Bitcoin	—	—	24209	1003	1105	205	0.0076	1	0.0040	38	0.0039
Pgp Arenas	—	—	24339	574	744	198	0.0210	2	0.0042	133	0.0019
Wiki Talk Nds	—	—	27455	45	149	256	0.0009	1	0.0006	3	0.0004
Openflights	—	—	30523	459	458	334	0.0618	3	0.0589	333	0.0069
Dimacs Polblogs	—	—	33453	606	605	313	0.2467	3	0.2419	313	0.0061
Gnutella	—	—	40017	421	227	61	0.1732	2	0.1037	30	0.0012
Ciaodvd	—	—	40156	978	1593	213	0.0224	1	0.0148	63	0.0089
Dblp Cite	—	—	49782	1634	1442	360	0.0142	1	0.0098	88	0.0047
Matter	—	—	58618	2283	3276	46	1.0359	2	1.0275	18	0.0064
Music	—	—	130457	63	81	23	0.0003	2	0.0001	13	0.0002
Appliances	—	—	2128628	1289	421	132	0.0041	1	0.0022	13	0.0015
Jawiki	—	—	3100520	5	5	23	0.0002	1	0.0001	23	0.0000
Eswiki	—	—	4896779	5	5	23	0.0002	1	0.0001	23	0.0000

Algorithm 4 Branch and Pivot

```

 $D^* \leftarrow \text{THETAHEURISTIC}(G)$ 
function BRANCHP( $S = (U_S, V_S), C = (U_C, V_C)$ )
  if  $C = \emptyset$  then
    if  $|E(S)| > |E(D^*)|$  and  $|U_S| \geq \theta$  and  $|V_S| \geq \theta$  then
       $D^* \leftarrow S$ 
    return
   $C_0 \leftarrow \{v \in C \mid \bar{d}_S(v) = 0\}$ 
  if  $C_0 = \emptyset$  or  $|C \setminus C_0| \geq k - |\bar{E}(S)|$  then
     $u \leftarrow \operatorname{argmax}_{v \in C} \bar{d}_S$ 
     $(S', C') \leftarrow \text{UPDATE}(S, C, u)$ 
    BRANCHP( $S' \cup \{u\}, C'$ )
    BRANCHP( $S, C \setminus \{u\}$ )
  else
     $u \leftarrow \operatorname{argmin}_{v \in C_0} \bar{d}_C$ 
    if  $\bar{d}_C(u) > k - |\bar{E}(S)| > 0$  then
       $(S', C') \leftarrow \text{UPDATE}(S, C, u)$ 
      BRANCHP( $S' \cup \{u\}, C'$ )
      BRANCHP( $S, C \setminus \{u\}$ )
    else
      for  $v \in \{u\} \cup N_C(u)$  do
         $(S', C') \leftarrow \text{UPDATE}(S, C, v)$ 
        BRANCHP( $S' \cup \{v\}, C'$ )
         $C \leftarrow C \setminus \{v\}$ 

```
